

**Universidade Cruzeiro do Sul Braz Cubas**

**Ryan Catão de Paula 742651786**

**Isaque Rodrigues Valim 43992960**

**SISTEMA DE GERENCIAMENTO DE ACADEMIA**

Mogi das Cruzes

2026

**Ryan Catão de Paula 742651786**

**Isaque Rodrigues Valim 43992960**

**Sistema de Gerenciamento da academia**

Proposta de Projeto RFC apresentada à  
disciplina de Estruturas de Dados, sob  
orientação de Andrea Ono Sakai .

Mogi Das Cruzes

2026

## SUMÁRIO

|     |                                        |
|-----|----------------------------------------|
| 1   | NOME DO SISTEMA .....                  |
| 2   | PROBLEMA .....                         |
| 2.1 | Descrição do Problema .....            |
| 2.2 | Quem Enfrenta Esse Problema? .....     |
| 2.3 | Frequência e Impacto .....             |
| 2.4 | Importância da Solução .....           |
| 3   | SOLUÇÃO PROPOSTA .....                 |
| 3.1 | Visão Geral do Sistema .....           |
| 3.2 | Fluxo Principal da Aplicação .....     |
| 3.3 | Módulos Principais .....               |
| 3.4 | Interação do Usuário .....             |
| 4   | USO DAS ESTRUTURAS DE DADOS .....      |
| 4.1 | Lista .....                            |
| 4.2 | Fila .....                             |
| 4.3 | Pilha .....                            |
| 5   | COMPLEXIDADE COMPUTACIONAL .....       |
| 5.1 | Operações Custosas Identificadas ..... |
| 5.2 | Análise de Escalabilidade .....        |
| 5.3 | Dashboard de Desempenho .....          |
| 6   | CONCLUSÃO .....                        |
| 6.1 | Resumo da Proposta .....               |
| 6.2 | Principais Benefícios .....            |
| 6.3 | Viabilidade Técnica .....              |
| 7   | CHECKLIST DE AUTOAVALIAÇÃO .....       |
| 8   | REFERÊNCIAS .....                      |

## **1 NOME DO SISTEMA**

Sistema de Gerenciamento de Academia

## **2 PROBLEMA**

### **2.1 Descrição do Problema**

Academias lidam com um grande volume de dados relacionados a alunos, planos, professores e aulas. Em muitos casos, esses dados são organizados de forma ineficiente, resultando em lentidão nas buscas e dificuldades no gerenciamento das informações.

### **2.2 Quem Enfrenta Esse Problema?**

A equipe de recepção, que precisa localizar rapidamente os dados dos alunos, e os próprios alunos, que enfrentam filas e demora no atendimento.

### **2.3 Frequência e Impacto**

O problema ocorre constantemente, especialmente em horários de pico. O impacto inclui atrasos no atendimento, insatisfação dos clientes e dificuldade no controle de planos ativos e inativos.

### **2.4 Importância da Solução**

Resolver esse problema melhora a eficiência do atendimento, reduz filas e garante melhor organização dos dados, impactando diretamente na experiência do cliente e no faturamento da academia.

## **3 SOLUÇÃO PROPOSTA**

### **3.1 Visão Geral do Sistema**

O sistema tem como objetivo gerenciar alunos, professores, planos e aulas utilizando estruturas de dados eficientes, garantindo rapidez no armazenamento e na busca das informações.

### **3.2 Fluxo Principal da Aplicação**

1. **Cadastro e recepção de alunos:** Cadastro de aluno na recepção com coleta de dados e definição do plano escolhido.

2. **Armazenamento estruturado:** Armazenamento dos dados em listas independentes separadas por domínio conceitual (Lista de Alunos, Lista de Planos, Lista de Professores).
3. **Gestão de Exceções:** Tratamento de cenários complexos de negócio, como cancelamento de planos, trancamento de matrículas e alunos inadimplentes.
4. **Organização contínua:** Organização e ordenação constante dos dados nas listas para garantir a eficiência das buscas futuras.
5. **Controle de Fluxo Físico:** Gerenciamento de múltiplas filas simultâneas, controlando tanto o atendimento geral na recepção quanto o limite de vagas em aulas específicas.
6. **Recuperação de Dados:** Busca rápida por dados (utilizando busca sequencial ou binária) para liberação de catraca ou consultas da recepção.
7. **Exibição e Histórico:** Exibição do resultado ao usuário e registro da ação no histórico do sistema.

### 3.3 Módulos Principais

*Módulo 1:* Cadastro de alunos — responsável por inserir e armazenar dados dos alunos

*Módulo 2:* Gerenciamento de planos — controle de planos ativos e inativos

*Módulo 3:* Controle de aulas — organização das aulas disponíveis

*Módulo 4:* Sistema de atendimento — controle de filas

*Módulo 5:* Histórico de ações — registro de operações realizadas

### 3.4 Interação do Usuário

O usuário pode cadastrar alunos, buscar por nome ou ID, remover registros, listar alunos, consultar planos e participar de filas de atendimento ou aulas.

## 4 USO DAS ESTRUTURAS DE DADOS

### 4.1 Lista

## O que é armazenado na lista?

Para evitar uma modelagem de dados simplificada e difícil de manter, o sistema utilizará listas separadas e independentes: uma Lista de Alunos, uma Lista de Professores, uma Lista de Planos e uma Lista de Aulas.

## Por que usar lista?

- *Precisa manter ordem de inserção*
- *Precisa acessar elementos por posição*
- *Precisa percorrer todos os elementos*
- *Precisa inserir/remover em posições específicas*
- *Outro motivo: No domínio de uma academia, a lista é a estrutura ideal para catalogar cadastros permanentes. Ela permite que a recepção percorra sequencialmente os registros durante auditorias de planos ativos/inativos e possibilita o acesso direto aos dados de um aluno específico através do seu índice na estrutura.*

## Operações Principais

*Inserção:* - *Onde:* no final da lista - *Quando:* ao cadastrar novos alunos.

*Remoção:* - *Onde:* em qualquer posição da lista - *Quando:* ao excluir um aluno.

*Busca:* - *Critério:* por nome ou ID - *Quando:* ao procurar um aluno específico no sistema.

*Travessia:* - *Finalidade:* para listar todos os alunos.

## Exemplo Concreto

Lista contendo todos os alunos cadastrados no sistema.

## 4.2 Fila

### O que é enfileirado?

Alunos aguardando liberação na recepção ou vagas em aulas.

**Cenários Avançados de Fila:** O sistema lidará com múltiplas filas operando simultaneamente. Existirá uma Fila de Atendimento Geral na recepção para dúvidas e pagamentos, e Filas de Espera exclusivas e independentes para cada aula que atingir sua lotação máxima (ex: Fila de Espera para o Spinning das 19h).

### **Por que usar fila (FIFO)?**

Porque o atendimento deve respeitar a ordem de chegada.

### **Operações Principais**

*Enqueue (enfileirar):* - *O que é enfileirado:* alunos aguardando atendimento ou vaga em aulas - *Quando acontece:* quando um aluno entra na fila de atendimento ou de espera.

*Dequeue (desenfileirar):* - *O que é desenfileirado:* o primeiro aluno da fila - *Quando acontece:* quando o aluno é atendido ou liberado para a aula.

*Peek (espiar):* - *Para que serve:* visualizar o próximo aluno a ser atendido sem removê-lo da fila.

### **Por que FIFO é essencial neste caso?**

Garantir justiça no atendimento. Sem isso, alunos poderiam ser atendidos fora de ordem.

### **Exemplo Concreto**

Fila de alunos aguardando atendimento na recepção.

## **4.3 Pilha**

### **O que é empilhado?**

Histórico de ações do sistema.

### **Por que usar pilha (LIFO)?**

Para permitir desfazer ações recentes.

### **Operações Principais**

*Push (empilhar):* - *O que é empilhado:* ações realizadas no sistema (como cadastro, remoção ou edição de alunos) - *Quando acontece:* sempre que uma nova ação é executada no sistema.

*Pop (desempilhar):* - *O que é desempilhado:* a última ação realizada - *Quando acontece:* quando o usuário deseja desfazer uma ação.

*Peek (espiar topo):* - *Para que serve:* Para que serve: visualizar a última ação realizada sem removê-la da pilha.

### Por que LIFO é essencial neste caso?

A última ação realizada deve ser a primeira a ser desfeita.

### Exemplo Concreto

Um recepcionista altera erroneamente o plano de um aluno de "Anual" para "Mensal". Utilizando o histórico em pilha, ele pode acionar a função "Desfazer" para reverter essa modificação específica e restaurar o plano correto imediatamente.

### Limitações da Pilha

Para evitar o consumo excessivo de memória do sistema, a pilha de histórico terá uma limitação de tamanho, armazenando apenas as últimas 10 ações do usuário. Ações mais antigas que isso não poderão ser desfeitas por essa estrutura.

## 5 COMPLEXIDADE COMPUTACIONAL

### 5.1 Operações Custosas Identificadas

- a) Busca sequencial
- b) Remoção de elementos
- c) Ordenação da lista

### 5.2 Análise de Escalabilidade

| Operação         | Estrutura | Complexidade | Justificativa                                                                                       | Impacto (100k)                                                    |
|------------------|-----------|--------------|-----------------------------------------------------------------------------------------------------|-------------------------------------------------------------------|
| Inserção         | Lista     | $O(1)$       | Inserção no final                                                                                   | Tempo de execução imperceptível, instantâneo.                     |
| Busca sequencial | Lista     | $O(n)$       | Percorre todos elementos                                                                            | Gargalo severo; lentidão que pode causar travamento na interface. |
| Busca binária    | Lista     | $O(\log n)$  | Divide o conjunto. Pré-condição vital: A lista deve estar obrigatoriamente ordenada antes da busca. | Alta eficiência; tempo de resposta na casa de milissegundos.      |



|                   |       |        |                                           |                                                                  |
|-------------------|-------|--------|-------------------------------------------|------------------------------------------------------------------|
| Remoção           | Lista | $O(n)$ | Reorganiza elementos                      | Atraso moderado perceptível durante a realocação dos índices.    |
| Enqueue / Dequeue | Fila  | $O(1)$ | Operações ocorrem apenas nas extremidades | Escalabilidade perfeita; sem impacto mesmo em horários de pico.  |
| Push / Pop        | Pilha | $O(1)$ | Operações ocorrem apenas no topo          | Escalabilidade perfeita; desfazimentos ocorrem instantaneamente. |

### 5.3 Dashboard de Desempenho

#### Métricas de Fila:

- Tempo médio de atendimento: tempo médio que um aluno leva para ser atendido.
- Tamanho da fila: quantidade de alunos aguardando atendimento.
- Tempo de espera: tempo médio que um aluno permanece na fila.

#### Métricas de Lista:

- Quantidade de alunos: total de alunos cadastrados no sistema.
- Tempo de busca: tempo médio para localizar um aluno.
- Número de registros: quantidade total de dados armazenado.

#### Métricas de Pilha:

- Quantidade de ações registradas: total de operações armazenadas no histórico.
- Número de desfazer realizados: quantidade de ações revertidas.
- Tamanho da pilha: quantidade atual de ações armazenadas.

#### Alertas de Performance:

- Fila muito longa: quando a quantidade de alunos na fila ultrapassa um limite definido.
- Tempo de busca elevado: quando o tempo de busca excede o esperado.
- Sistema operando normalmente: quando todos os indicadores estão dentro dos padrões ideais.

## 6 CONCLUSÃO

### 6.1 Resumo da Proposta

O sistema de gerenciamento de academia foi desenvolvido com o objetivo de organizar e otimizar o controle de alunos, planos, professores e aulas. Utilizando estruturas de dados como listas, filas e pilhas, o sistema melhora a eficiência das operações e reduz o tempo de atendimento. Além disso, ele proporciona maior controle sobre as informações e facilita a busca de dados. Dessa forma, a solução contribui para um funcionamento mais ágil e organizado da academia.

### 6.2 Principais Benefícios

1. Redução no tempo de atendimento aos alunos
2. Melhor organização e controle das informações
3. Maior eficiência na busca e gerenciamento de dados

### 6.3 Viabilidade Técnica

O sistema possui **viabilidade técnica média**, pois exige conhecimento em programação e estruturas de dados, mas pode ser desenvolvido utilizando linguagens acessíveis como Python. As funcionalidades propostas são simples de implementar e não dependem de tecnologias complexas, tornando o projeto viável para estudantes e aplicações reais de pequeno a médio porte.

## 7 CHECKLIST DE AUTOAVALIAÇÃO

Antes de entregar, verifique se você:

### 7.1 Clareza e Viabilidade

- [X] O problema está claramente definido
- [X] A solução é viável e faz sentido
- [X] O fluxo do sistema está claro

### 7.2 Uso das Estruturas de Dados

- [X] Lista é usada de forma justificada (não forçada)
- [X] Fila representa um processo FIFO real
- [X] Pilha representa um comportamento LIFO real
- [X] Todas as operações estão bem explicadas

### **7.3 Análise de Complexidade**

- [X] Operações custosas foram identificadas
- [X] Complexidade de cada operação está correta
- [X] Análise de escalabilidade é realista
- [X] Dashboard de desempenho é relevante

### **7.4 Qualidade da Escrita**

- [X] Texto claro e objetivo
- [X] Sem erros gramaticais
- [X] Exemplos concretos fornecidos
- [X] Todas as seções foram preenchidas

## **8 REFERÊNCIAS**

CORMEN, Thomas H. et al. *Algoritmos: teoria e prática*. 3. ed. Rio de Janeiro: Elsevier, 2012.

GOODRICH, Michael T.; TAMASSIA, Roberto; GOLDWASSER, Michael H. *Estruturas de dados e algoritmos em Python*. Porto Alegre: Bookman, 2013.

PYTHON SOFTWARE FOUNDATION. *Python Documentation*. Disponível em: <https://docs.python.org/3/>. Acesso em: 22 abr. 2026.